Bibliographic Report: An Agentic Framework for IaC Misconfiguration Analysis and Remediation

Ahmedou Yahye Kheyri

Supervised by: Hala Bezine

March 7, 2026

Abstract

Infrastructure as Code (IaC) has become a central practice in the deployment and management of IT infrastructures. However, IaC scripts are prone to configuration errors that can lead to critical security vulnerabilities. Traditional Retrieval-Augmented Generation (RAG) systems provide security advice but cannot verify the applicability or effectiveness of the proposed solutions. This bibliographic report explores recent work in the field of IaC misconfiguration detection and remediation, focusing on approaches based on machine learning and Large Language Models (LLMs). It then proposes an architecture for an agentic framework capable not only of proposing fixes but also of validating them using specialized external tools (such as Checkov, GLITCH) and providing actionable, verified corrections. The keywords for this study are: RAG System, IaC Misconfigurations, Agentic Framework.

Contents

1 Introduction **2**

2 State of the Art **2**
2.1 Taxonomies of Defects and Security Smells 2
2.2 Defect Detection: From Metrics to Semantic Models 3
2.3 Specialized Tools for Security Analysis 3
2.4 Enhancing LLMs for Secure Code Generation 3
2.5 LLM vs Static Analyzer Comparison 3
2.6 Lessons for an Agentic Framework 3

3 Proposed Agentic Framework **4**
3.1 Central Agent (Orchestrator) 5
3.2 Contextual Analyzer 5
3.3 Knowledge Retriever (RAG) 5
3.4 Fix Generator (Fine-tuned LLM) 5
3.5 External Tool Integrator (Validator) 5
3.6 Patch Formatter and Explanation Generator 5
3.7 General Workflow 5

4 Conclusion and Perspectives 6

1 Introduction

The widespread adoption of DevOps practices has made Infrastructure as Code (IaC) a key element for automated infrastructure management. Tools like Ansible, Terraform, Puppet, and Chef allow the desired state of systems to be described in configuration files. However, these scripts can contain security flaws — known as *security smells* — such as plaintext passwords, overly permissive permissions, or outdated dependencies [1, 2]. Major incidents (financial losses, service outages) have demonstrated the need for early detection and reliable correction of these issues [3].

Classical detection approaches rely on static analyzers (linters) based on predefined rules. More recently, machine learning techniques and Large Language Models (LLMs) have been explored for a finer semantic understanding of code [4, 5]. Concurrently, RAG (Retrieval-Augmented Generation) systems allow enriching an LLM's responses with external knowledge (vulnerability databases, documentation). However, a standard RAG system remains a simple "answer provider": it gives recommendations without being able to verify for itself whether they are correct or truly applicable to the analyzed code. For infrastructure security, this lack of verification constitutes a major risk.

This bibliographic report aims to lay the foundations for an **agentic framework** that transforms the passive RAG system into an **active and critical agent**. Such an agent must be capable of:

1. Validating its own recommendations using self-verification mechanisms and specialized security tools (Checkov, GLITCH, etc.).

2. Proposing iterative and contextual correction strategies, going beyond simple information retrieval.

3. Integrating external validators for cross-verification independent of the language model.

4. Generating concrete, ready-to-use fixes, accompanied by clear technical explanations.

The remainder of this report is organized as follows: Section 2 presents a structured state of the art around defect taxonomies, detection techniques, existing tools, and LLM vs static analyzer comparisons. Section 3 describes the proposed architecture for the agentic framework. Finally, Section 4 concludes and outlines research perspectives.

2 State of the Art

2.1 Taxonomies of Defects and Security Smells

A fundamental first step for error correction is their classification. Rahman *et al.* [6] proposed a taxonomy of eight defect categories in IaC scripts (called the "Gang of Eight"), later validated on modern tools like Pulumi and AWS CDK [7]. This taxonomy includes categories such as *Configuration Data*, *Dependency*, and *Security*.

Furthermore, Rahman *et al.* [1] identified seven specific *security smells* (hard-coded passwords, weak cryptography, etc.). A recent extension by War *et al.* [8] expanded this number to **62 categories** by analyzing seven different IaC tools (Ansible, Terraform, Chef, Puppet, Pulumi, SaltStack, Vagrant). This new taxonomy, aligned with the CWE classification, provides a rich knowledge base for a RAG system.

2.2 Defect Detection: From Metrics to Semantic Models

Early defect prediction approaches used product and process metrics. Dalla Palma *et al.* [9] built prediction models for Ansible using Random Forest, showing that structural metrics (number of tokens, text entropy) are more predictive than process metrics.

More recently, deep learning models and LLMs have been employed for finer-grained detection. War *et al.* [4] used CodeBERT and LongFormer to capture the semantics of IaC scripts, significantly improving precision compared to traditional methods (from 0.46 to 0.92 for Ansible). De Vito *et al.* [5] proposed SecLLM, a framework based on prompt engineering that outperforms the GLITCH tool on several datasets, while managing LLM non-determinism through caching and confidence score filtering.

2.3 Specialized Tools for Security Analysis

Several dedicated tools have been developed for detecting flaws in IaC. GLITCH [10] is an agnostic framework that transforms scripts (Ansible, Chef, Puppet) into an intermediate representation to apply detection rules for nine smells, with better precision than SLIC and SLAC. Other tools like Checkov, Terrascan, or Ansible-Lint are widely used in the industry.

2.4 Enhancing LLMs for Secure Code Generation

The problem of LLMs generating insecure code was addressed by Li *et al.* [11] with GenSIAC, an instruction-tuning dataset that improves the security awareness of models (increasing F1-score from 0.3 to 0.85). This approach shows that more reliable fixes can be obtained directly from the LLM.

2.5 LLM vs Static Analyzer Comparison

A recent comparative evaluation [12] pitted three static analyzers (SonarQube, CodeQL, SnykCode) against three LLMs (GPT-4.1, Mistral Large, DeepSeek V3). LLMs achieved better average F1-scores (0.75–0.80 vs 0.26–0.55) due to their contextual reasoning ability, but they produced more false positives and poorly localized errors. This study highlights the value of a hybrid approach combining the best of both worlds.

2.6 Lessons for an Agentic Framework

The bibliographic analysis highlights several key points:

- An extensive smell taxonomy is available to populate a knowledge base.
- LLMs (especially after fine-tuning) can generate relevant fixes.
- Specialized tools like GLITCH can serve as reliable external validators.

- Hybrid approaches (LLM + analyzers) are promising for combining recall and precision.

However, no existing system autonomously integrates fix generation, external validation, and an iterative critique loop. This is the objective of our proposal.

3 Proposed Agentic Framework

We propose an architecture composed of six interconnected modules, each addressing part of the tasks defined in the introduction.

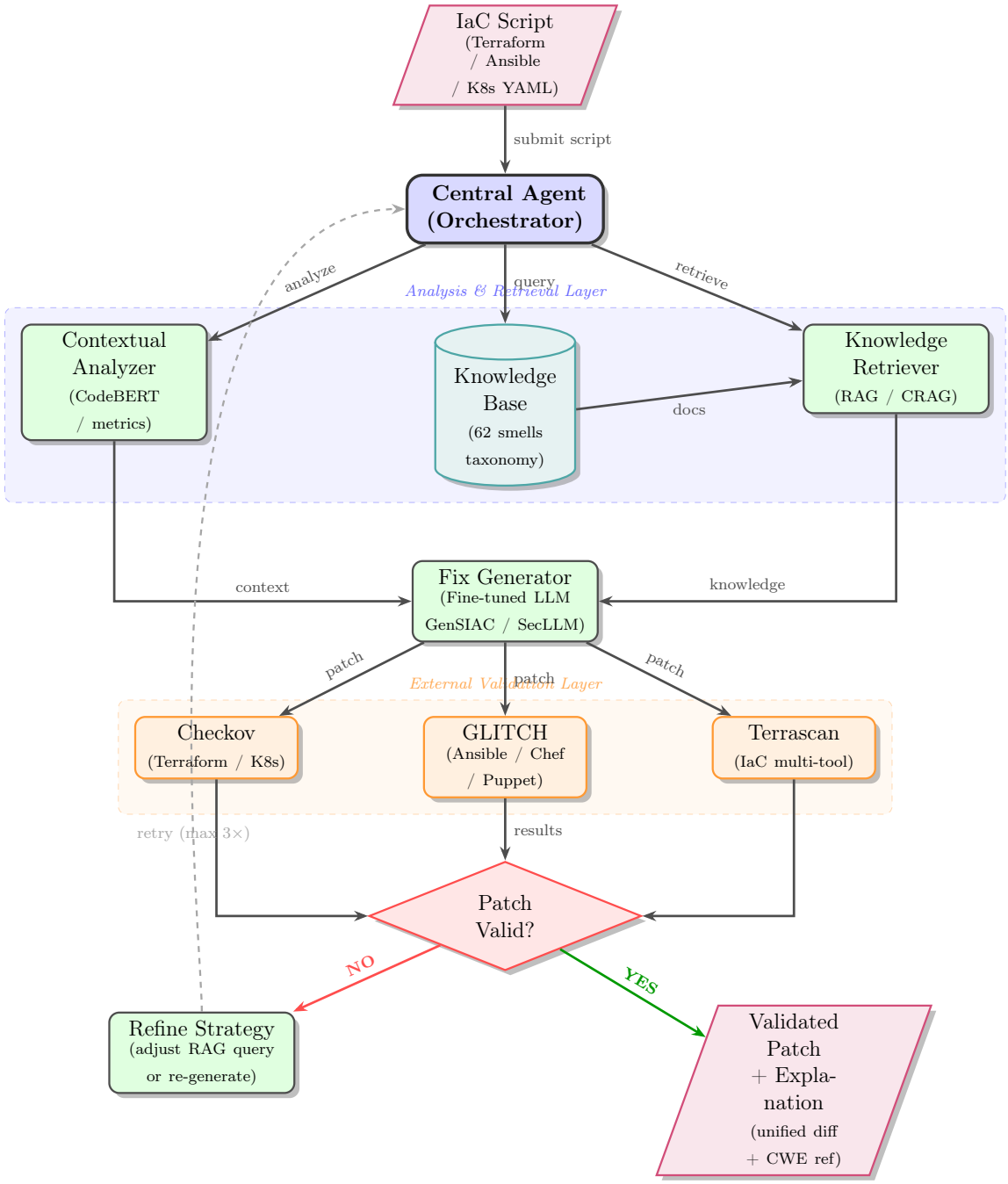

Figure 1: Detailed architecture of the proposed agentic framework for IaC misconfiguration remediation.

3.1 Central Agent (Orchestrator)

This is the heart of the system. It receives the IaC script as input, manages the workflow, maintains the state of the various analyses, and makes decisions (e.g., launch a validation, iterate on a fix, etc.). It relies on the results of the other modules to converge towards a validated solution.

3.2 Contextual Analyzer

This module analyzes the script to identify the IaC tool type (Terraform, Ansible, etc.), extract structural metrics (lines of code, complexity, etc.), and pinpoint risk areas using lightweight models (e.g., CodeBERT [4] or the metrics from [9]). It provides enriched context to the fix generator.

3.3 Knowledge Retriever (RAG)

A vector knowledge base is built from existing taxonomies [8], audit reports (as in [13]), and documentation on fixes. This module retrieves the most relevant information based on the context (error type, technology) and provides it to the LLM to guide patch generation.

3.4 Fix Generator (Fine-tuned LLM)

This component uses an LLM specialized in generating secure IaC code, ideally fine-tuned on a dataset like GenSIAC [11]. It receives the original code, context from the analyzer, and relevant documents from the RAG, and produces one or more patch proposals. A confidence score is associated with each proposal (inspired by SecLLM [5]).

3.5 External Tool Integrator (Validator)

This is the key module for validation. It takes the proposed patch and automatically runs several recognized static analysis tools: GLITCH [10], Checkov, Terrascan, etc. It compares the results on the original code and the patched code to ensure the smell has disappeared and no new issues have been introduced. This step provides validation independent of the LLM.

3.6 Patch Formatter and Explanation Generator

Once the patch is validated, this module produces a unified diff file ready to be applied by the developer. It also generates a technical explanation in natural language, describing the corrected flaw (with CWE reference if possible) and justifying the remediation choices.

3.7 General Workflow

1. The user submits an IaC script.
2. The contextual analyzer characterizes it and detects candidates for correction.
3. For each candidate, the RAG provides information about the smell and examples of fixes.

4. The LLM generator proposes one or more patches.
5. The external tool integrator validates each patch.
6. If a patch is validated, it is formatted and presented to the user. Otherwise, the agent can iterate by modifying the RAG query or requesting a new generation.

4 Conclusion and Perspectives

This bibliographic report has highlighted the richness of recent work in the field of Infrastructure as Code security. It has also shown the limitations of passive RAG systems and the need to move towards an active agent capable of validating its proposals. The proposed architecture integrates the best approaches from the literature (specialized LLMs, knowledge bases, validation tools) to provide a trusted assistant for developers.

Future work will consist of implementing a prototype of this framework, building a knowledge base from existing taxonomies and audit reports, and evaluating its performance on real-world datasets (such as those used in the cited studies). Special attention will be paid to managing LLM non-determinism (caching, confidence filtering) and integrating a feedback loop with the user to refine corrections.

References

[1] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. ICSE*, 2019, pp. 164–175.

[2] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security smells in ansible and chef scripts: A replication study," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 1, pp. 1–31, 2021.

[3] A. War, A. Diallo, A. Habib, J. Klein, and T. F. Bissyande, "Vulnerabilities in infrastructure as code: what, how many, and who?" *Empirical Software Engineering*, vol. 30, no. 120, 2025.

[4] A. War, A. A. Rawass, A. K. Kabore, J. Samhi, J. Klein, and T. F. Bissyande, "Detection of security smells in iac scripts through semantics-aware code and language processing," 2025, preprint.

[5] G. De Vito, F. Palomba, and F. Ferrucci, "SecLLM: Enhancing security smell detection in iac with large language models," *IEEE Access*, 2025.

[6] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proc. ICSE*, 2020, pp. 752–764.

[7] W. Oliveira, F. Paiva, T. E. Pereira, and J. Brunet, "A defect taxonomy for infrastructure as code: A replication study," 2025, preprint.

[8] A. War, S. L. B. Nikiema, J. Samhi, J. Klein, and T. F. Bissyande, "Security smells in infrastructure as code scripts: A taxonomy update beyond the seven sins," 2025, preprint.

[9] S. Dalla Palma, D. Di Nucci, F. Palomba, and D. A. Tamburri, "Within-project defect prediction of infrastructure-as-code using product and process metrics," *IEEE Trans. Softw. Eng.*, vol. 48, no. 6, pp. 2086–2104, 2022.

[10] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proc. ASE*, 2022, pp. 1–12.

[11] Y. Li, M. Grella, D. Nahmias, G. Engelberg, D. Klein, G. Guizzardi, T. van Ede, and A. Continella, "GenSIAC: Toward security-aware infrastructure-as-code generation with large language models," 2025, preprint.

[12] D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," 2025, preprint.

[13] J. Chen, Y. Shen, J. Zhang, Z. Li, J. Grundy, Z. Shao, Y. Wang, J. Wang, T. Chen, and Z. Zheng, "FORGE: An LLM-driven framework for large-scale smart contract vulnerability dataset construction," 2025, preprint.